

Mecha-Basketball 3D

Technical Report and User Manual

Alessandro Carotenuto

MSc Artificial Intelligence & Robotics, Sapienza University of Rome

July 12, 2026

Interactive Graphics, Prof. Marco Schaerf

Dept. of Computer, Control and Management Engineering (DIAG)

Repository

[https:](https://github.com/SapienzaInteractiveGraphicsCourse/final-project-intentionallyblankname)

[//github.com/SapienzaInteractiveGraphicsCourse/final-project-intentionallyblankname](https://github.com/SapienzaInteractiveGraphicsCourse/final-project-intentionallyblankname)

Live demo

[https:](https://sapienzainteractivegraphicscourse.github.io/final-project-intentionallyblankname/)

[//sapienzainteractivegraphicscourse.github.io/final-project-intentionallyblankname/](https://sapienzainteractivegraphicscourse.github.io/final-project-intentionallyblankname/)

Contents

1 Project Overview

Mecha-Basketball 3D is a real-time basketball match between procedurally generated robots, rendered on a GLTF court with Three.js. The player picks a robot class, each with its own stats and special move, then controls movement, dribbling, aiming and shooting directly; nothing is a scripted replay.

No external model is used for the robots: each class is built in Three.js from primitives, organized as hierarchical `THREE.Group` trees and animated procedurally. Only two assets are external: the court and the ball (both GLTF, Section ??). All three classes are fully playable by the human player and by the enemy AI, and are swappable from the Main Menu with no page reload. Every class shares one stat struct, `{ speed, shooting, steal, block }`, driving both gameplay and the comparison bars on the selection screen.

Class	Locomotion	Special Move	Spd	Sht	Stl	Blk
Mobile Manipulator (AMR)	Wheels	Dash	3	1	3	3
Legged Manipulator	Legs (trot gait)	Jump	2	3	2	5
Drone	Flight (rotors)	Flight (grab-and-rise)	5	2	1	1

Table 1: Robot roster and stats (scale 1–5, Shooting 1–3).

Two modes: **Practice** (one robot, empty court) and **1v1** (AI opponent with tactical navigation, defense, steal and block; first to 11 points wins).

2 Project Structure

The codebase is organized by domain under `src/`. `main.js` is the only top-level file: it owns the scene, the render loop and every piece of global state, and wires the other modules together

(Section ??).

```
src/
  main.js          scene, camera, lights, render loop, HUD wiring
  SharedEnums.js   frozen enums: GameMode, Team, TimeOfDay, ROBOT_KEYS
  robots/
    RobotBase.js    shared FSM, stats, locomotion/aim hooks
    AMRManipulator.js  wheeled class, Dash
    LeggedManipulator.js  legged class, Jump
    Drone.js         flying class, Flight
    ModelMakers/     procedural geometry factories (one per class)
  gameplay/
    Basketball.js    ball wrapper, HANDLED / FREE_SHOT / FREE
    BallPossession.js  dribble state machine, handling, pickup
    ShootingSystem.js  shot physics, hoop assist, scoring, preview
    CollisionWorld.js  sphere-vs-AABB, sphere-vs-torus
    CombatMoves.js    steal, block
    EnemyAI.js        1v1 tactical state machine
  ui/MainMenu.js     menu navigation, gameplay reset
  debug/             tuning panel, collision wireframe overlays
  audio/SoundEffects.js  synthesized Web Audio sound effects
  utils/             math helpers, physical constants
public/models/       court + ball GLTF, the only external assets
```

3 Development Environment, Libraries and Assets

Built with **Three.js** (~0.178.0) on **Vite** (~6.0.0), plain JavaScript ES modules, no TypeScript or framework. Three.js was chosen over raw WebGL to spend effort on modelling and animation rather than shader/buffer boilerplate. `vite.config.js` sets `base: './'` so the bundle works from a GitHub Pages subpath.

The renderer targets soft, physically plausible shadows: `PCFSoftShadowMap` (VSM was tried and is too slow at 4096²), ACES filmic tone mapping, sRGB output.

Beyond the Three.js core, only official addons are used: `GLTFLoader`, `PointerLockControls`, the `EffectComposer` post-processing stack (Section ??) and the procedural Sky skydome. No physics engine and no animation library (`tween.js`, `AnimationMixer`) are integrated: collision response, the dribble state machine and every joint animation are hand-written inside the render loop.

Two external, pre-made assets:

- **Court** (Sketchfab, modified): a custom `GLTFLoader` plugin re-implements `KHR_materials_pbrSpecularGL` (removed from Three.js r152+); some material colors were hand-edited in the raw `scene.gltf`; line z-fighting fixed with `polygonOffset`.
- **Basketball**: a dedicated GLTF with color, normal and metalness/roughness maps.

Everything else, all three robots (geometry, materials, textures), the skybox and every sound effect, is generated procedurally at runtime.

4 Architecture and Recurring Patterns

`main.js` owns all shared mutable state (`manipulator`, `basketball`, `cameraState`, ...) and is the only file allowed to import from every other module; every gameplay module receives what it needs through a single *context object* instead of importing from `main.js`, which would create circular dependencies:

```
main.js
```

```

const gameContext = { camera, scene, sfx, controls, cameraState, ... }

initBallPossession({ ...gameContext, dribbleState, shootingState })
initShootingSystem({ ...gameContext, getManipulator, getShotDirection })
initCombatMoves   ({ ...gameContext, stealState, blockState })
initMainMenu      ({ ...gameContext, ... })

```

each init returns { updateX, triggerY, ... }, called back from animate()

A live reference such as the active robot cannot be a plain captured value: it changes at run-time when the player swaps class, and `basketball` itself is null until its GLTF finishes loading asynchronously. Both are passed as zero-argument *getter functions* (`getManipulator()`, `getBasketball()`) instead, so every consumer always reads the current value.

The render loop drives two independent dispatch chains, one per side (player, enemy):

```

animate()
  updateTimeOfDayTransition()
  updateShootAnimation()           windup / release / recover
  updateBallDispatch(player)
    pickup active?      -> updatePickup()
    shot released?      -> updateShotFlight() + checkForPickup()
    no ball?            -> checkForPickup()
    else                -> updateHandling() or updateDribble() (fixed-step)
  [1v1] updateEnemyAI() -> triggerShoot / triggerSteal / triggerBlock
  [1v1] updateBallDispatch(enemy)
  updateSteal() / updateBlock()    both robots
  updateBallSpin()
  updateTrajectoryPreview()
  composer.render()

```

Four patterns recur across the codebase:

- **Finite state machines everywhere:** robot state (DRIBBLE/HANDLING/NO_BALL), ball state (HANDLED/FREE_SHOT/FREE), the dribble (push/drop/rise), the shot (windup/release/recover) and combat moves (reach/resolve) are all plain objects with a `phase` string and a `phaseT` timer, advanced by hand each frame. Enums are frozen plain objects (`Object.freeze`), JavaScript has no native enum.
- **Fixed-timestep accumulator:** the dribble integrates at a constant 120 Hz and the shot flight at 240 Hz, decoupled from the variable render `delta`, via an accumulator (Section ??).
- **Scratch objects:** every hot-path function (collision resolution, trajectory preview, ball spin) reuses `THREE.Vector3/Box3` instances allocated once, instead of allocating per call; the trajectory preview alone can call the collision resolver up to 2400× per frame while aiming.
- **Composition over inheritance for robots:** `RobotBase` is a class that wraps a plain factory function (`Object.assign(this, factory())`), keeping procedural geometry construction (never touched once stable) separate from gameplay state (stats, FSM, special moves).

5 Hierarchical Models

All three robot classes carry the same 3-revolute-joint arm above (or, for the Drone, hanging below) their locomotion base:

base (yaw) → link1 → elbow (pitch) → link2 (tapered) → wrist (pitch) → end effector + paddle

each joint a nested `THREE.Group`, so rotating a parent (e.g. the elbow) carries every child with it: the standard scene-graph technique for an articulated model. `link2` uses a tapered cross-section

from a 4-sided `CylinderGeometry` (`radialSegments = 4` degenerates the cylinder into a square prism; different top/bottom radii turn it into a truncated pyramid). The ball’s resting point during handling is the convergence point of the two paddle plates’ normals, at distance $d/\cos(\alpha/2)$ from the wrist, not the paddle’s flat surface center, which visibly interpenetrated the ball.

Per-class hierarchy below the arm:

- **Mobile Manipulator:** 4 wheels (tori) under a disc chassis; the wheel group’s scale derives from the disc radius, so resizing the chassis keeps the wheels attached.
- **Legged Manipulator:** 4 two-joint legs (hip + knee) at the wheel offsets, each oriented radially outward; 25% larger overall. The flat foot pivots at its rear edge, the same off-center-pivot trick as the paddle.
- **Drone:** a body with four continuously spinning rotor arms (alternating spin direction between neighbors); the arm hangs upside down through a fixed flip group (`rotation.x =  $\pi$`) between the yaw base and the first link.

`RobotBase` wraps each procedural factory, owning the shared `RobotState` FSM, the stat-derived speed getter, and empty special-move hooks each subclass overrides.

TODO: figure. Main Menu robot selection screen, 3 cards with live 3D preview

Figure 1: Robot selection screen, with live-dribbling 3D previews and stat bars.

6 Materials, Lighting and Textures

6.1 Lights

1 `HemisphereLight` (warm ambient, approximating the original Sketchfab IBL), 1 `DirectionalLight` (4096² shadow map, frustum sized to cover the court), 4 `PointLight` (lamp posts) and 2 `SpotLight` (hoop fixtures, lit only at Sunset/Night; toggled via intensity, never `.visible`, to avoid a shadow-map allocation stutter the first time a light becomes visible). Colors, intensities and sun position follow a `TimeOfDay` preset table (Sunrise/Day/Sunset/Night), cross-fading over 2.5 s instead of snapping.

Scale. The court uses centimeter-scale units spanning thousands of units, so every physically-motivated Three.js default needed recalibration: the shadow frustum spans ± 2500 units, giving low texel density at 4096² (fixed with `shadow.bias/normalBias`); point-light intensity is candela with inverse-square decay, so lamp-to-ground values around 2×10^5 were needed at ~ 250 units; each lamp sits at the center of its own glass globe, self-shadowing unless `shadow.camera.near` exceeds the globe radius.

6.2 Post-Processing

`EffectComposer` pipeline: `RenderPass` $\rightarrow$ `SSAOPass` $\rightarrow$ `OutputPass` $\rightarrow$ `SMAAPass`. `SSAOPass.minDistance/maxDistance` are normalized fractions of the near-far depth range, not world units; `camera.near` was raised from 0.1 to 5 against `far = 5000` to stop depth precision from saturating; the kernel radius shrank from 90 to 12 to remove dark halos from surfaces reading as local occlusion behind a silhouette. `renderer.antialias` has no effect once rendering goes through the composer’s targets, so `SMAAPass` runs last instead.

6.3 Textures

Two kinds: **downloaded** (ball color map, court diffuse map) and **procedurally generated** (ball normal/metalness/roughness; robot, bench and lamp-post normal/roughness/bump maps, built at runtime from a height field).

6.4 Procedural Skybox

The flat background color was replaced with the **Sky** addon (Preetham atmospheric scattering, no HDR download). The sun direction in the sky is always derived from the real **DirectionalLight** position, never tuned independently, so the sky’s glow and the cast shadows stay aligned by construction. At Night the Preetham model degenerates (built for daylight scattering), so the sky mesh swaps to a flat dark color once the animated transition completes.

TODO: figure. close-up of a robot body showing procedural PBR texture detail

Figure 2: Procedurally generated normal/roughness detail on a robot’s chassis.

7 Animation

Every animation runs imperatively inside `requestAnimationFrame`; nothing is imported.

System	Technique
Automatic dribble	Push/drop/rise finite-state machine at fixed 120 Hz, synced with the elbow/link1 motion
Shot flight	Projectile motion under constant gravity, sub-stepped at 240 Hz, full collision response
Wheel steering	Exponential-smoothing angular interpolation, shortest path, wrap-safe
Legged gait	Diagonal-pair trot, advancing only while position actually changes frame to frame
Drone flight feel	Body bank and thrust-tilt composed via quaternion multiplication, not Euler angles, so the tilt axis stays attached to the nose at any yaw
Special moves	Dash (6×-speed burst, 4 s cooldown), Jump (parabolic arc, 5 s cooldown), Flight (grab-rise-hold-descend, rigid ball attachment)
Shot animation	Windup/release/recover pose, elbow coupled to camera pitch while aiming
Time-of-Day fades	2.5 s smoothstep cross-fade of every light/sky parameter
Ball spin	Derived each frame from real displacement (axis = $\hat{u} \times \vec{v}$, rate = $ \vec{v} /r$), consistent across every state with no per-state simulation

Table 2: Animated systems; static geometry is limited to the court and lamp posts.

Fixed timestep. Integrating the dribble on the variable render `delta` produced a real bug: release velocity, a single-frame finite difference, collapsed to near zero whenever a frame hitch landed exactly on the push-easing saturation frame, and the ball dropped dead instead of bouncing. A 1/120 s accumulator (a slow frame runs several steps, a fast one may run none) removes the pathological case and makes the trajectory deterministic regardless of frame rate.

Third-person camera. While dribbling, an orbit camera follows the robot (mouse yaw/pitch, `lookAt` pinned, pitch clamped to $\sim 3\text{--}51^\circ$). While aiming, it switches to a free orientation, yaw/pitch drive `camera.quaternion` directly, decoupled from position, allowing a look over the robot toward the hoop. Transitions always interpolate (position `lerp`, rotation `slerp`) toward a target recomputed every frame, since the two formulas are structurally different and switching without this layer snaps visibly.

8 Physics, Collisions and Scoring

No physics engine: every collision test is a closed-form formula, **sphere vs. AABB** (ball vs. backboard/walls/poles/benches, robot vs. static geometry) and **sphere vs. torus** (ball vs. rim), both using $\vec{v}' = \vec{v} - (1 + e)(\vec{v} \cdot \hat{n})\hat{n}$ with a per-material restitution e (backboard 0.15, rim 0.3,

walls/poles 0.55, benches 0.5). The 66 wall/bleacher boxes are extracted mesh by mesh from the court GLTF, not one aggregate box, since the venue has intentional gaps an aggregate would misrepresent.

Shot flight is sub-stepped at 240 Hz because at full speed a single frame can move the ball farther than the backboard is thick. Post-bounce collision cooldowns are tracked per object: a single global cooldown after a rim bounce would suspend the backboard check at the exact moment the deflected ball reaches it.

A hoop-assist potential field pulls an airborne shot toward the rim center, strength scaled by the shooter’s **shooting** stat. It is a clamped *position* correction, not an acceleration: an acceleration accumulates with time spent in the cone, flinging slow close-range shots past the rim, while a pull clamped at the center cannot overshoot.

Scoring: 2 points if the shooter was inside the three-point arc at the moment of release, 3 otherwise, arc radius read from the court GLTF’s accessor data. A basket is detected by interpolating the exact crossing point of the rim plane between two physics samples; testing only the sample already past the plane misses steep, close-range shots that genuinely went in.

9 Audio

Five sound effects, all synthesized at runtime with the Web Audio API, no audio file shipped: **score** (two ascending sine notes), **bounce** (low downward-sweeping tone plus a low-pass noise transient), **shoot** (band-pass noise sweeping upward), **steal** (same technique sweeping downward, shorter), **block** (low-pass noise hit over a descending tone). Each has a short linear attack before its exponential decay; an instant jump to peak gain produces an audible click. Background music is not yet implemented (Section ??).

10 User Interaction and Controls

The Main Menu (HTML/CSS overlaid on the canvas) walks through GAMEMODES → ROBOT selection (a second opponent-selection screen in 1v1) → TIME OF DAY + START. All three classes are instantiated (hidden) at page load, so switching class is a visibility toggle, never a reload. Selection cards run the same dribble state machine as the real game on a live offscreen renderer.

Input	Effect
P	Debug panel: per-component geometry tuning, live camera readout
M	Toggle Spectate (free-fly) / Play (third-person)
Mouse (Play)	Orbits the camera; aims the arm
WASD (Play)	Move relative to camera facing; wheels/legs steer toward it
Left Shift (Play)	Special move for the active class
Right mouse, held (Play)	HANDLING: pauses the dribble, frees the camera to aim
Left click (HANDLING)	Shoot: crosshair raycast direction, constant speed, 60% inside the arc
Walk near a loose ball	Automatic pickup, no key
Q / E (1v1, no ball)	Steal / Block
Esc (Play)	Pause menu
1–8	Toggle collision/contact wireframe overlays

Table 3: Full control scheme.

Shot strength is constant rather than charge-based: varying force at aim time would change the previewed trajectory while lining it up. A live trajectory preview (a 3D tube rebuilt every frame from the same physics as the real flight) shows exactly where the shot will go, turning green when it crosses the hoop.

The debug panel exposes every tunable parameter as live sliders, plus a Copy Config button serializing the current geometry; the shipped default dimensions were tuned this way. An Animation Preview section can force any pose for inspection from a free camera angle.

TODO: figure. debug panel open, showing per-component sliders

Figure 3: Debug panel, used to tune every robot’s geometry live.

11 1v1 Mode and Enemy AI

The opponent runs a small tactical state machine, `EnemyState` $\in \{\text{CHASE_BALL}, \text{ATTACK}, \text{DEFEND}\}$, recomputed every frame from ball possession: chase a loose ball, attack toward its hoop and shoot in range, or position between the player and the player’s hoop while attempting Steal or Block. Shot elevation is solved with the ballistic-projectile equation at fixed speed, falling back to a 50° arc if the target is unreachable.

Steal and Block share one module between player and AI: a reach/sweep animation, a bounding-box contact test (asymmetric for Steal, wide margin in front of the sweeping robot, almost none behind), and a resolve step re-validating possession at the end of the sweep rather than at first contact. Robot-vs-robot overlap resolves with a possession-based asymmetric correction (the ball holder yields 25% of the separation, the other robot 75%); a robot mid-shot is never displaced, since its position also drives the chase camera.

12 Course Material Correspondence

Table ?? maps specific claims from the course slides to their implementation.

Lecture	Slide content	Implementation
19, Physics	Collision response, coefficient of restitution r	<code>CollisionWorld.js</code> : $\vec{v}' = \vec{v} - (1+e)(\vec{v} \cdot \hat{n})\hat{n}$ generalizes the slide’s floor-only formula to any surface normal
19, Physics	“If not interested in realistic response, do position correction”	<code>clampRobotToCourt</code> , hoop assist: position-only corrections, no velocity change
19, Physics	“Position and velocity form the simulation state”	<code>dribbleState.ballVelocityY</code> , <code>shotVelocity</code> , each paired with a position
19, Physics	Semi-implicit Euler integration	<code>stepDribble/stepShotFlight</code> : velocity updated first, then position with the new velocity
19, Physics	Explicit Euler error scales with the square of the timestep	Motivates the fixed small steps, 1/120 s dribble, 1/240 s shot
19, Physics	Gravity as the simplest constant force	<code>BALL_GRAVITY</code> , the only force in the shot flight
11, Shading	Phong specular reflection, perfect mirror direction	Same reflection geometry as $e = 1$ in the collision formula
21, Three.js	WebGL boilerplate vs. Three.js	Direct motivation for choosing Three.js over raw WebGL (Section ??)

Table 4: Course material referenced by the implementation.

13 How to Run and Deploy

The live build is hosted on GitHub Pages: <https://sapienzainteractivegraphicscourse.github.io/final-project-intentionallyblankname/>, no installation required.

To run locally:

```
git clone https://github.com/SapienzaInteractiveGraphicsCourse/final-project-intentionallyblankname.git
cd final-project-intentionallyblankname
npm install
npm run dev # -> localhost:5173
```

Deployment is automatic: a GitHub Actions workflow (`.github/workflows/deploy.yml`) builds the Vite bundle and publishes `dist/` to Pages on every push to `main`.

14 Conclusions and Limitations

Table ?? maps the course’s mandatory requirements to this report.

Requirement	Where	Section
Hierarchical model, structural animation	Three procedural robots, nested-Group hierarchy; steering, aiming, dribbling and special moves animate through it	???
At least one light	1 hemisphere, 1 directional (shadowed), 4 point, 2 spot	???
Textures of two kinds	Ball: color + normal + metalness/roughness; robots/benches/lamps: procedural normal/roughness/bump	???
Real user interaction	Full match controls, Main Menu, pause, options	???
Most objects animated, all in JavaScript	Every robot, the ball, the shot, the sky and the lights	???

Table 5: Coverage of the course’s mandatory requirements.

The two choices that shaped the project most: building everything procedurally (robots, textures, skybox, sounds; only the court and ball are external), and simulating on fixed timesteps decoupled from the render rate. A recurring lesson, met independently in the shadow setup, SSAO configuration and light intensities, is that a graphics library’s physically-motivated defaults assume a scene scale; importing an asset with different units means re-deriving each such parameter rather than tuning it blindly.

Known limitations: no 3v3 mode; no background music, only the synthesized SFX of Section ??; robot color customization does not persist across a page refresh, in-memory for the session only.

[TODO] personal conclusions and retrospective, in the student’s own words.